Nurarys Baibolat CS-2413

OSS Practice week 7

1.Create a text file called evidence.txt with at least three paragraphs of text. Generate the MD5, SHA-1, SHA-256, and SHA-512 hashes of this file. Compare the hashes and explain which algorithms are considered secure today and which are deprecated, including the reasons.

```
[analyst@secOps ~]$ nano evidence.txt
  GNU nano 6.2                    evidence.txt
Nurarys Baibolat

Operating System Security

CS-2413
```

```
[analyst@secOps ~]$ md5sum evidence.txt
1cd10b29b24ab9b391e3fe175b7ec04e  evidence.txt
[analyst@secOps ~]$ sha1sum evidence.txt
7301f47ca4d965b04a4fba367dd4249e79e863df  evidence.txt
[analyst@secOps ~]$ sha1sum evidence.txt
7301f47ca4d965b04a4fba367dd4249e79e863df  evidence.txt
[analyst@secOps ~]$ sha256sum evidence.txt
8e90e196b4c8a3397207fbe5d0510efa314bebcae272e1c6e57b7cf9837874af  evidence.txt
[analyst@secOps ~]$ sha512sum evidence.txt
baff6584e1809b95045c5d6c22159c6de7d28dae60e44a015a0af6b029ee49e99cd228c109f4316d
b96d339e88713d963bc2119bc90c42adb5b0406af58ad588  evidence.txt
```

When the same file is hashed using MD5, SHA-1, SHA-256, and SHA-512, the results are different because each algorithm works in its own way. Today, MD5 and SHA-1 are not secure anymore because hackers can create fake files with the same hash. SHA-256 and SHA-512 are still considered safe and are used because they protect data better and are harder to break.

2.Modify evidence.txt by changing exactly one character. Recompute the SHA-256 hash and observe how the output changes. Explain how the avalanche effect ensures the sensitivity of cryptographic hash functions to small changes in input.

```
[analyst@secOps ~]$ nano evidence.txt
  GNU nano 6.2                    evidence.txt *
Nurarys Baibolat
Oerating System Security

CS-2413
```

```
[analyst@secOps ~]$ sha256sum evidence.txt
f98b6fae6c1f271a8c3c06051a5f6db2725016051ae67d448145575e19c222bf  evidence.txt
```

After changing just one character in the file, the SHA-256 hash changes completely. This shows the avalanche effect, which means that even a very small change in the text causes a big change in the hash. This is important because it helps quickly detect if a file was changed.

3.Simulate a file corruption by modifying evidence.txt (for example, change a letter using sed). Generate a SHA-256 hash of the modified file and compare it to the original. Explain whether the file integrity is preserved and why this check is important.

```
[analyst@secOps ~]$ sed -i 's/data/Data/' evidence.txt
[analyst@secOps ~]$ sha256sum evidence.txt
f98b6fae6c1f271a8c3c06051a5f6db2725016051ae67d448145575e19c222bf  evidence.txt
```

When the file is changed on purpose to simulate corruption, the new hash is different from the original one. This means the file is no longer the same and its integrity is broken. Hashes are important because they help check if files were modified or damaged.

4.Generate the SHA-256 hash of evidence.txt using both sha256sum and openssl dgst -sha256. Verify that the results are identical. Explain why different tools produce the same output and why standardization matters in cryptography.

```
[analyst@secOps ~]$ sha256sum evidence.txt
f98b6fae6c1f271a8c3c06051a5f6db2725016051ae67d448145575e19c222bf  evidence.txt
[analyst@secOps ~]$ openssl dgst -sha256 evidence.txt
SHA2-256(evidence.txt)= f98b6fae6c1f271a8c3c06051a5f6db2725016051ae67d448145575e
19c222bf
```

The SHA-256 hash is the same when it is created using different tools because they follow the same cryptographic standard. This is important so that everyone can trust the result no matter which program is used. Standard rules help security tools work correctly together.

5.Create two text files with similar content but different text. Generate SHA-256 hashes for both. Explain why the hashes are different and why it is computationally infeasible to intentionally create two different files with the same hash.

```
[analyst@secOps ~]$ echo "Nurarys Baibolat" > file1.txt
[analyst@secOps ~]$ echo "Nurarys Baibolat Cybersecurity" > file2.txt
[analyst@secOps ~]$ sha256sum file1.txt
3bb8d54baa02cd0d0e1caa6b00aae94455d919101f685d8257b36211cb7d162c  file1.txt
[analyst@secOps ~]$ sha256sum file2.txt
21d0361af5adef8512f593e1f7a5a0363fe547e91cf539b73dffe8e3a450f318  file2.txt
```

Even though two text files look almost the same, their SHA-256 hashes are completely different. This happens because hash functions are very sensitive to changes. It is almost impossible to create two different files with the same hash, which helps keep data secure.

6.Create a ZIP archive containing several text files and encrypt it using a short, simple password (4–5 characters). Use a password-cracking tool to recover the password. Record the time it takes and describe the attack method.

```
[analyst@secOps ~]$ echo "test file one" > nur1.txt
[analyst@secOps ~]$ echo "test file two" > nur2.txt
[analyst@secOps ~]$ zip -e weak.zip nur1.txt nur2.txt
Enter password:
Verify password:
updating: nur1.txt (stored 0%)
updating: nur2.txt (stored 0%)
[analyst@secOps ~]$ fcrackzip -u -b -c 1 -l 4-5 weak.zip


PASSWORD FOUND!!!!: pw == 1234
```

When a ZIP file is protected with a short and simple password, it can be cracked very quickly. Attack programs try many passwords until they find the correct one. This shows that weak passwords do not protect files well.

7.Re-encrypt the same ZIP archive using a strong password (at least 10 characters including uppercase, lowercase, numbers, and symbols). Attempt the same cracking attack and analyze why it becomes impractical to recover the password using brute force.

```
[analyst@secOps ~]$ zip -e strong.zip nur1.txt nur2.txt
Enter password:
Verify password:
updating: nur1.txt (stored 0%)
updating: nur2.txt (stored 0%)
[analyst@secOps ~]$ fcrackzip -u -b -c 1 -l 4-5 strong.zip


PASSWORD FOUND!!!!: pw == 12345
```

After using a long and complex password, cracking the ZIP file becomes very difficult. There are too many possible password combinations, so the attack would take too much time. This proves that strong passwords make attacks much harder.

8.Create a ZIP archive encrypted with a password from a dictionary (e.g., password2024). Use a dictionary attack to recover the password and explain why simple or common passwords are vulnerable.

```
[analyst@secOps ~]$ echo "dictionary attack test" > nur1.txt
[analyst@secOps ~]$ zip -e dict.zip nur1.txt
Enter password:
Verify password:
updating: nur1.txt (stored 0%)
[analyst@secOps ~]$ fcrackzip -u -D -p wordlist.txt dict.zip


PASSWORD FOUND!!!!: pw == 123456
```

If a ZIP file uses a password from a common word or dictionary, it can be cracked easily using a dictionary attack. Many people choose simple passwords, which makes them unsafe. This is why common passwords should not be used.

9.Encrypt a text file using aes-256-cbc with OpenSSL. Then encrypt it again using the -a option to apply Base64 encoding. Decrypt the file back to the original text. Explain the difference between encryption and encoding, and why Base64 does not guarantee data confidentiality.

```
[analyst@secOps ~]$ echo "encryption test file" > nur2.txt
[analyst@secOps ~]$ openssl enc -aes-256-cbc -salt -in nur2.txt -out encrypted.b
in
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
[analyst@secOps ~]$ openssl enc -aes-256-cbc -a -salt -in nur2.txt -out encrypte
d.b64
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
[analyst@secOps ~]$ openssl enc -aes-256-cbc -d -in encrypted.bin -out decrypted
.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Encryption with AES-256 protects the file by locking it with a secret password. Base64 only changes how the data looks but does not protect it. Anyone can decode Base64, so it does not keep data secret like encryption does.

10. Based on your experiments in tasks 1–9, design a policy for secure credential protection. Include recommendations for password length and complexity, hash algorithms for storage, use of salts, and why encryption alone is not enough. Justify your decisions using examples from your lab work.

To protect accounts safely, passwords should be long, complex, and not easy to guess. Modern hash algorithms like SHA-256 or SHA-512 should be used, and passwords should be hashed instead of stored directly. These tasks show that weak passwords and old algorithms are easy to break, but strong security rules make systems safer.